Progress Report — Jyotir

AMS 560 · Big Data Systems, Algorithms and Networks · Fall 2026 · Group 20

1. Objective and Project Description

We are building a **change-aware, feasibility-first scheduler** for Large Language Model (LLM) inference across geo-distributed GPU nodes. Current production schedulers treat each request independently and ignore dynamic hardware state (GPU temperature, power headroom, KV-cache capacity). Our system uses a two-stage pipeline: a **feasibility filter** that eliminates nodes which physically cannot serve a request, followed by a **contextual bandit policy** (sliding-window LinUCB) that learns which feasible nodes produce the best latency and energy outcomes over time.

The primary metric is **goodput** — requests per second finishing within the 250 ms TTFT SLO — and **adaptation time** — how quickly goodput recovers to ≥90% of its new steady-state after a cluster change event. We test across six scenarios: stable operation (S1), traffic burst (S2), electricity price shift (S3), thermal throttle (S4), network degradation (S5), and node outage + recovery (S6).

My role (Jyotir): Design and implement the feasibility filter and five static baseline scheduling policies. The filter is the first component every request hits. The baselines serve as the lower-bound comparison for the learned policy.

2. Timeline and Expected Deliverables

Week	Deliverable	Due
1	Feasibility filter interface + config file Stub objects (fake_node.py, stub_policy.py) for independent development	Week 1
2	Full feasibility filter (5 rules, YAML config, warming node support) All 5 baseline policies (Round-Robin, Min RTT, Least Load, Lowest Energy, Static Weighted) Optional: Power-of-Two baseline	Week 2
3	Edge-case test suite (all 6 test categories, 27 tests) Report section: Section 3 — Feasibility Filter and Baselines WORK_LOG.md documenting all files	Week 3
4	Swap stubs for real interfaces (Parikshit telemetry, Krupesh router) Support Riya's experiment runner integration	Week 4

3. Completion Status per Deliverable

Deliverable	Status	Notes
Feasibility filter (filter.py, feasibility.yaml)	**Complete**	5 rules: status, model, KV cache, power, temperature. Token-exact memory check. W
Stub objects (fake_node.py, stub_policy.py)	**Complete**	Allows building and testing without waiting for teammates. Swap is one import change
Round-Robin baseline	**Complete**	Rotates by index, handles single candidate.
Min RTT baseline	**Complete**	Per-region RTT lookup with average fallback.
Least Load baseline	**Complete**	Picks lowest queue_len + running.
Lowest Energy baseline	**Complete**	Cheapest price; excludes saturated nodes first.
Static Weighted baseline	**Complete**	Min-max normalised RTT + load + price, fixed weights 0.5 / 0.3 / 0.2.
Power-of-Two baseline (optional)	**Complete**	Random 2-sample, picks less busy.
Edge-case test suite (27 tests)	**Complete**	6 categories; all 27 pass under pytest.
Report section — Section 3	**Complete**	MD + PDF in report_section/; covers filter, all baselines, S1–S6 tables, test coverage.

WORK_LOG.md	**Complete**	Start-to-end log of all files with purpose and code summaries.
Real interface integration (Week 4)	**In Progress**	Waiting for Parikshit telemetry + Krupesh router to stabilise. Stubs ready to swap.

4. Plan to Complete Remaining Deliverables

Real interface integration (Week 4 target). All my code currently runs against stub objects. The integration task has two steps:

- **Parikshit's telemetry:** Replace `fake_node.py` imports with live node snapshots from Parikshit's simulator. The filter already handles the `kv_capacity_tokens` field that his updated schema provides, so no filter logic needs to change — only the import path.
- **Krupesh's router:** Replace `stub_policy.Policy` with Krupesh's `Policy` base class and confirm the `choose(request, candidates)` and `update(request, node_id, outcome)` signatures match. The baselines are already structured to this interface.

Once both interfaces are confirmed stable, the swap is a two-line change per baseline file (update the import, inherit the real base class). Estimated time: 2–3 hours after teammates confirm their interfaces are frozen.

5. Difficulties

Independent development without full team integration. My filter and baselines sit at the boundary of three teammates' work (Parikshit's telemetry, Krupesh's router, Sur's policy). I resolved this with a stub pattern — synthetic node and request objects that match the agreed schema — allowing me to build and test everything independently. The remaining risk is that a schema change late in the project requires re-running tests against the new field names.

Token-aware KV cache check. The initial filter used only a fill-fraction threshold (`kv_used_frac ≥ 0.90`). This was identified as insufficient: a request needing 1,050 tokens on a 1,000-token-capacity node at 50% fill would pass the fraction check but would be rejected by the GPU at runtime. The fix required adding a token-exact check when `kv_capacity_tokens` is available from Parikshit's telemetry, with the fraction check retained as a fallback.

Warming node safety gap. An early version of the filter fast-pathed warming nodes past the model, memory, power, and temperature checks (a bug). The fix restructured `_check_node` so the status check only determines reachability; all five rules run for warming nodes, and the warming tag is applied only if all five pass. Three new test classes were added to verify this behaviour.

All Week 1–3 deliverables are complete and pushed to the group repository. Week 4 integration is blocked only on teammate interface freeze.